

Company C – Jaeger Industries

Group 24 Specs Document

0 – Introduction

This document was developed by students at Michigan State University in CSE498, the capstone project of the Computer Science degree program. It will detail all implementation specifications for all classes and modules developed by Group 24 of Company C.

1 – C++ Classes

This section will detail the implementation and design of the C++ Web Interface classes created for this project. They are designed to allow a C++ programmer to change the underlying HTML and interface with the HTML DOM from their C++ code.

1.1 – WebImage (Sadwal (Luca) Patel)

1.1.1 Class Description

A class to manage an HTML image from your C++ code. It should simplify image usage in the program, handling asset loading and allowing a programmer to select size, position, alt text, etc, through C++ functions.

At a high level, WebImage should:

- Create an image element and attach it to a parent container/layout
- Load an image from an asset path or URL
- Allow the programmer to set size/position and accessibility fields like alt text
- Provide a basic loaded vs failed status so the rest of the UI can respond

1.1.2 Similar Classes in Standard Library

Some classes for interface choices -

- `std::string` / `std::string_view` (for things like ids, paths or alt text)
- `std::optional` (optional size or position that may be unset)
- `std::function` (might need for load / error callbacks)
- `std::runtime_error`, `std::invalid_argument` (if using exceptions and such)

1.1.3 Key Functions

Attachment Functions -

- Create or attach an image to a DOM parent container

- Attach/Detach (or auto-attach on construction)

Core Image Functions :

- Set/Get source like asset path
- Set for alt text
- Set size like the width or the height
- Set position or set layout
- Show or Hide visibility

For Loading :

- Query Load status (success/fail)
- If success/failure , register something so that we can show something up on the interface

Minimal styling (optional but useful)

- Set a small set of presentation fields (e.g., z-index) or provide a generic

1.1.4 Error Conditions

1- Programmer error - assert

- Arguments are invalid (non positive width or height , etc)
- Invalid ID of parent container / invalid DOM attachment assumptions
- Using the object when the state is invalid (if enforcing a particular state)

2 - Potentially recoverable - exception

- Failure to initialize required web binding
- Any internal resource failures (not able to create underlying DOM resources through our bridge)

3 - User/ environment error - status return / error flag

- Image fails to load due to bad path/URL , missing asset, network or browser restrictions. WebImage should expose “failed” so the UI can notify the user or go into some kind of contingency.

1.1.5 Expected Challenges

- Integration of DOM and Emscripten : deciding a similar approach for all Web classes
- Async loading : image load events happen later; we must handle callbacks safely.
- Layout coordination: deciding whether WebImage directly controls pixel position or WebLayout takes care of the layout.
- Testing : a small web demo and unit tests for non-DOM logic and error handling.

1.1.6 Coordination with Other Groups

Within the group:

- WebLayout: parenting + positioning conventions (coordinate system, attachments)
- WebCanvas: only if we later want the image to be drawn on the canvas

- WebButton / WebTextbox : any shared DOM utilities and consistent styling conventions

Across Company C :

- Any group defining asset defining / path conventions so that image id/name is consistent
- Any group defining what the interface is gonna display

1.2 – WebTextbox (Priyam Khanal)

1.2.1 Class Description

A class to manage an HTML text block from your C++ code, including font and format information.

1.2.2 Similar Classes in Standard Library

`std::string` – for content storage and manipulation

`std::unordered_map<std::string, std::string>` – for storing CSS properties/styles

`std::function` – for potential callbacks (if text events are triggered)

1.2.3 Key Functions

Possible functions:

Attachment Functions:

- Create or attach a text element to a DOM parent container
- Attach/Detach (or auto-attach on construction)

Content Functions:

- Set/Get text content
- Append text
- Clear text

Font & Typography:

- Set font family, size, weight, style
- Set line height

Styling Functions:

- Set text color
- Set background color
- Set text alignment (left, center, right)
- Set text decoration (underline, bold, etc.)

Layout Functions:

- Set position (x, y)
- Set size (width, height)
- Set padding/margin
- Set word wrap behavior

Visibility & State:

- Show/Hide visibility
- Set opacity

1.2.4 Error Conditions

Programmer Error (assert):

- Bad element ID (empty, missing, or reused)
- Negative width/height passed in
- Invalid CSS color value

Recoverable Error (exception):

- Parent DOM node not found or not ready yet
- Failed to create/attach the text element due to environment/runtime issues

User/Design issue (log or return):

- Text overflow (content exceeds container) — log a warning; caller can adjust size/wrap
- Style applied before element exists — return a failure flag or hit an assert per build mode

1.2.5 Expected Challenges

- Learning EM_ASM and type conversions
- Element/parent may not exist when we attach
- Keeping C++ state and DOM in agreement
- Fonts/spacing differ across browsers
- Frequent DOM updates across C++/JS boundary
- Hard to unit test DOM

1.2.6 Coordination with Other Groups

- WebLayout (Group 24): agree on ownership (who creates/destroys), positioning rules, and parent container IDs
- WebButton/WebImage (Group 24): share a small styling utility to avoid duplicate CSS code
- Event/callback patterns (Group 24): align on how elements register and dispatch events
- Data/Logging teams: confirm update frequency and any formatting expectations if text shows live stats/logs
- Database (Group 9): align on simple string formats if pulling text from stored records

1.3 – WebButton (Tess Gonda)

1.3.1 Class Description

A web button that will call a designated function when clicked. Provides a C++ interface for HTML button elements, enabling interactive UI controls with customizable appearance and behavior. Handles button creation, styling, state management, and event callbacks.

1.3.2 Similar Classes in Standard Library

- `std::function` for callback management

1.3.3 Key Functions

Basic Constructor:

- Default constructor
- Constructor with label
- Constructor with label and callback

Button Label Management:

- `SetLabel`
- `GetLabel`

Event Handling:

- `SetOnClick` - Tells button what to do when someone clicks it
- `Click()` - Trigger button programmatically

State Management:

- `SetEnabled/IsEnabled`
- `SetPressed/IsPressed`
- Prevent invalid operations like users clicking Submit twice

Styling Functions:

- Size control
 - Width and Height setters
- Color control
 - `SetColor/SetBackgroundColor/etc.`
 - Different states (normal, hover, pressed) need different colors

DOM Interaction:

- `AppendTo`
 - Buttons need to be added to the page
 - Different buttons go in different containers
- `Remove`
 - Clean up when button is no longer needed

1.3.4 Error Conditions

1. **Programmer Error**
 - a. Negative width/height
 - b. Operating on deleted button element
 - c. Invalid parent element ID
2. **Potentially Recoverable Error**
 - a. Callback function throws during execution

- b. DOM manipulation fails
- 3. **User Error**
 - a. Clicking a disabled button

1.3.5 Expected Challenges

- Ensuring callbacks still remain valid when a button is destroyed or a C++ object goes out of scope
- Properly handling hover/active/disabled states
- Preventing quick repeated clicks from causing problems
- Handling desktop and mobile interactions
- Making sure JavaScript event listeners are removed properly when buttons are destroyed

1.3.6 Coordination with Other Groups

- **Group 6 - ActionMap:** Buttons will trigger actions registered in ActionMap
- **Group 6 - BehaviorTree:** Buttons to trigger or interrupt agent behaviors
- **Group 7 - StateGrid:** Buttons for placing/removing world elements
- **Group 8 - EventQueue:** User button clicks may schedule events
- **Group 9:** Buttons for database operations (post/like/comment/etc., depending on what we choose for the company project)
- **Group 23 - ActionLog:** Button clicks should be logged for replay functionality

1.4 – WebCanvas (Naod Ghebredngl)

1.4.1 Class Description

A class to manage an HTML5 canvas object, allowing more dynamic drawing to the screen.

1.4.2 Similar Classes in Standard Library

There aren't many similar classes that are helpful but `std::vector` to store objects to draw is a good one

1.4.3 Key Functions

- Basic constructor
Creates the canvas element
- Constructor with size parameters
Allows specifying size of objects
- Text and Image rendering.
- Canvas cycling for different images
- State management for colors and specificities to objects
- Basic shape drawing functions (rectangle, square, triangle)

1.4.4 Error Conditions

- Invalid size dimensions
- Drawing before creating canvas
- Attempting to draw outside bounds

1.4.5 Expected Challenges

- Learning and utilizing Emscripten in the right way
- Ensuring canvas updates regularly and is visually responsive and efficient

1.4.6 Coordination with Other Groups

WebCanvas will coordinate primarily with the agent and world simulation groups to represent the visual aspect of objects. Also with WebLayout and other UI groups.

1.5 – WebLayout (Abigail MacKersie)

1.5.1 Class Description

A class to manage the layout of other web objects on the screen, directly interfacing with the HTML DOM. It will be a tree-structured container to both read and write to the HTML tags, attributes, and elements.

1.5.2 Similar Classes in Standard Library

There does not currently exist a tree-structured object in the C++ Standard Library, however, there are multiple ways to implement a Node-like structure using STL classes such as the *std::vector* combined with smart pointers for object ownership and lifetimes.

1.5.3 Key Functions

Construction/Destruction:

- An HTML parser to construct the DOM Layout object upon initialization
 - The constructor will produce a tree-structured container of the information contained within the DOM, including the root document, the tags, the header, the body, and all content contained within those sections.
- A robust destructor that deletes all Nodes so that data is freed when going out of scope

Traversal and Reading:

- A function that uses standard algorithms to step through the layers of the DOM architecture. Should allow the user to find any tags, elements, or div sections by keyword, ID, or other identifying features.
 - This function will return all instances of an element or tag that matches the search input criteria in a *std::vector*. A vector of size 0 will indicate failure or a lack of elements that match the criteria.
- A function that allows the user to read the key-value pairs for the attributes on a given element

Adding Elements and Editing Elements:

- A function that creates an element at the given position in the hierarchy (defaults to the end) of a given divider. It will both be inserted into the C++ DOM vector and placed within the DOM text to be reflected within the HTML of the application.
- A function that edits a given element's attributes
- A function that edits a given element's contents
- A function that edits a given element's position within the section divider

1.5.4 Error Conditions

Programmer Error:

- Duplication of elements
- Failure to handle deletes within the DOM recreation
- Not freeing pointers from memory
- Incorrect traversals of the *std::vector*

Potentially Recoverable Error:

- If there is an error within the HTML code (e.g. an unclosed element, an attribute that is incorrectly assigned, etc.) or some other error that would prevent the HTML from rendering correctly, the class will throw an error requesting user intervention.

User Error:

- If the HTML document cannot be opened or read from in plain text, the class will throw an error about invalid input.
- If the user attempts to interact with a deleted or otherwise not present element within the HTML that is still present in the C++ DOM recreation, the class will throw an error requesting a review of the discrepancies.

1.5.5 Expected Challenges

The class will likely require other smaller supporting classes to be produced to minimize the code within the WebLayout class itself. It will be expected to fulfill a large array of roles flawlessly for the other classes to be able to correctly interact with the HTML DOM that lies beneath. It will likely also require thoughtful implementation and rigorous testing to ensure that core functionality is maintained throughout the development process.

1.5.6 Coordination with Other Groups

This class will likely need communication with the Database and Data Analytics groups to determine what information will need to be placed within the DOM as attributes and as non-standard elements.

2 – Module

The Web Interface module will provide the primary user interaction layer for Company C's project of choice, whether it's a campus simulation, collaborative world-building system, or a social media platform. The interface will handle two tasks: rendering the world state so users can see what's happening, and capturing user input to allow for interaction with agents and the environment. We expect that we'll need to display spatial information (such as where things are in the world) and text-based information (messages, posts, agent info, etc.), so our class design includes canvas-based rendering as well as more traditional UI elements.